OBERON

AI-Assisted Earth Observation Change Intelligence

An open-source, self-hostable API that turns public satellite imagery into evidence-backed change findings

Product thesis
Images are evidence. Oberon uses geospatial AI to automate the expensive part: finding, ranking, localizing, and classifying meaningful surface changes across many places and dates.

Document	Value
Document type	Product Brief v3 - AI-Centered
Date	June 19, 2026
Author	Farzin Shifat
Stage	Idea stage - pre-MVP
Technical feasibility	Supported by existing open data, STAC/COG standards, and Clay tooling
User demand	Unvalidated - requires design partners
Differentiation	Partially validated - narrow AI inference and provenance layer
Business model	Hypothesis - OSS core, hosted convenience, enterprise support

1. Executive Summary

Oberon is a proposed open-source, self-hostable Earth observation analysis API. A user supplies an area and two time windows. Oberon selects suitable public satellite observations, creates aligned analysis-ready imagery, uses a geospatial foundation model to identify likely changes, validates those findings against deterministic spectral measurements, and returns structured results with imagery, geometry, model version, data quality, and provenance.

The first product is not a generic "ask the Earth anything" interface. It is a focused change-intelligence service:

> **Initial user job**
> "Show me where this area changed between two periods, rank the most meaningful changes, and provide enough evidence for a person or another system to verify them."

Why AI belongs in the product

Returning imagery alone is useful for one-off inspection. AI becomes valuable when the user must monitor hundreds or thousands of areas, when changes are subtle across spectral bands, or when software needs polygons, rankings, and categories rather than pictures. The model is used for representation, localization, and classification. It is not used for coordinate math, source discovery, cloud masking, or provenance.

MVP promise

- Self-hostable REST API and CLI for before/after analysis over Sentinel-2 Level-2A imagery.
- Patch-level AI change map based on Clay feature maps or embeddings.
- Hybrid verification using spectral deltas such as vegetation, water, and burn-related indices.
- Structured GeoJSON findings with source scenes, acquisition dates, quality flags, model version, and thumbnails.
- Optional task-specific classifier only after an evaluated dataset exists.

What is deliberately not promised

- Arbitrary natural-language answers about any location.
- A production-ready air-gapped defense system.
- A single Rust binary containing the complete PyTorch model stack.
- Calibrated probabilities before task-specific evaluation and calibration.
- A universal satellite protocol or operator network during the MVP.

2. Problem and Product Wedge

The practical problem

Public Earth observation data is increasingly discoverable through standards such as STAC, and cloud-native formats such as COG support partial reads. However, a developer still has to select suitable scenes, inspect local cloud quality, align observations from different dates, normalize sensor bands, tile large areas, run a model, and preserve enough provenance to make the output trustworthy. Existing frameworks solve important portions of this stack, but a deployable, opinionated AI change-analysis service is still a narrower product opportunity rather than an empty infrastructure category. [3][4][5][7]

The product wedge

Oberon should own one complete workflow:

Area + before window + after window
Return likely change polygons, ranked change scores, optional change-type suggestions, before/after imagery, deterministic evidence, and quality metadata.

Why this wedge is better than “land cover at a point”

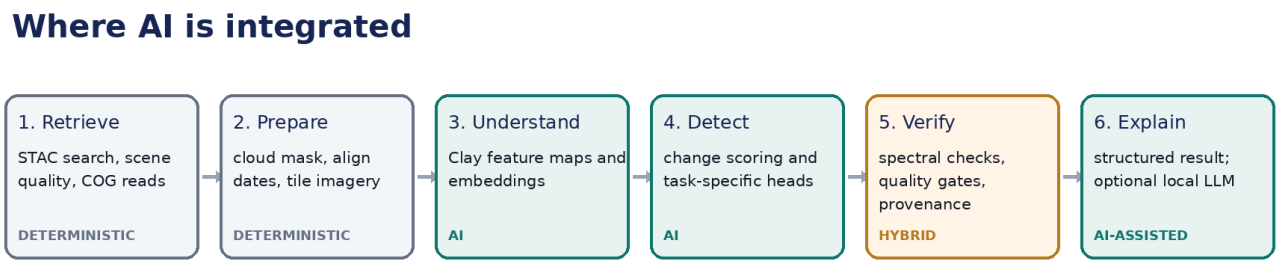
Criterion	Point land-cover query	Change intelligence
User value	Describes a place once	Monitors events and change over time
AI necessity	A map product may already answer it	AI reduces repeated human review
Output	One class label	Ranked regions, geometry, evidence, and review queue
Expansion	More labels	Flooding, vegetation loss, construction, burn scars, damage
Commercial path	Generic developer utility	Operational monitoring and alerts

Target users for discovery

- Climate and conservation teams monitoring vegetation loss, water movement, or restoration areas.
- Humanitarian and disaster-response teams prioritizing areas for review after floods or fires.
- Agtech teams monitoring large portfolios of fields where human inspection does not scale.
- Infrastructure and real-estate intelligence teams tracking construction or land disturbance.
- Researchers who need reproducible, self-hosted model inference and complete provenance.

Important boundary
Defense and intelligence may value local deployment, but they should not be the initial go-to-market assumption. Procurement, security, compliance, and disconnected-operation requirements are much larger than the MVP.

3. Where AI Is Integrated



AI component 1: geospatial representation

Clay receives multispectral imagery plus sensor, location, time, wavelength, and resolution metadata and produces learned feature maps and embeddings. Clay v1.5 supports multiple sensors, and the current Sentinel-2 example uses ten bands with 256 x 256 chips and produces 1024-dimensional embeddings. [1][2]

Benefit: the encoder can capture patterns that are difficult to describe with a single hand-built index and can transfer its learned representation into downstream classification, regression, segmentation, and change tasks. Clay documentation explicitly presents these downstream uses, but they still require a trained or fine-tuned task head. [1][8][9]

AI component 2: patch-level change localization

For each aligned before/after tile, Oberon extracts Clay feature maps and calculates patch-level representation differences. The resulting map identifies where the learned representation changed most. For the first MVP, this should be called a change score, not a probability. It is useful for triage even before a task-specific labeled model exists.

AI component 3: task-specific models

Once a use case and labeled dataset are selected, Oberon trains a small head on frozen or fine-tuned Clay features. Examples include flood segmentation, vegetation-loss classification, built-up expansion, or burn-scar detection. Clay publishes examples for classification, regression, and segmentation heads, which makes this technically credible. [8][9]

AI component 4: optional language interface

A local or user-provided language model may translate a natural-language request into a strict analysis plan and summarize the structured result. It must not invent measurements or serve as the image-analysis authority. The geospatial pipeline and task models remain the source of truth.

Feedback loop

Human review decisions can be stored as labels. Over time, these examples support active learning, region-specific calibration, and customer-specific task heads. This is where a real data moat could emerge, but only with explicit consent and clear data ownership.

4. Hybrid Intelligence: AI Plus Deterministic Checks

A production-quality Earth observation system should not treat a foundation model as an oracle. Oberon should combine learned features with established measurements and explicit data-quality gates.

Layer	Purpose	Example output
Scene quality	Reject unusable evidence	valid pixels, cloud/shadow fraction, date offset
Deterministic indices	Provide interpretable physical signals	NDVI delta, NDWI delta, NBR delta
Foundation model	Represent complex multispectral and spatial patterns	feature maps, embeddings, change distance
Task head	Translate features into a defined domain output	flood mask, vegetation-loss class
Calibration	Make scores decision-safe	thresholds, precision/recall, abstention
Provenance	Make every finding reproducible	scene IDs, bands, model and config versions

Confidence semantics

- General change score: a relative ranking produced by the representation-change method. It is not a probability.
- Class probability: allowed only for an evaluated task head with documented calibration.
- Quality score: reflects input suitability, not whether the semantic conclusion is correct.
- Abstain state: returned when clouds, seasonal mismatch, missing imagery, or model uncertainty make a conclusion unreliable.

Human-review design

The API should make uncertainty actionable. Results above a high threshold may enter an alert workflow. Medium-confidence findings should enter a review queue. Low-quality or out-of-distribution inputs should return an abstention rather than a confident-looking answer.

> **Trust principle**
> Oberon should always return the evidence used to produce the result. A user must be able to inspect the before image, after image, change overlay, source metadata, and model version.

5. Product Experience and API

Primary workflow

1. User submits a polygon or point with before and after time windows.
2. Oberon searches configured STAC catalogs and ranks candidate scenes.
3. The system reads only required COG windows, applies quality masks, aligns pixels, and creates analysis tiles.
4. Clay produces before/after features; Oberon generates a patch-level change map.
5. Deterministic metrics validate and help interpret the change.
6. An optional task head proposes a defined change category.
7. Oberon returns GeoJSON findings, artifacts, provenance, and review guidance.

Proposed request

```
POST /v1/change
{
  "geometry": {"type": "Polygon", "coordinates": [...]},
  "before": {"from": "2026-04-01", "to": "2026-04-15"},
  "after":  {"from": "2026-06-01", "to": "2026-06-15"},
  "task": "general_change",
  "max_cloud_fraction": 0.15
}
```

Proposed response

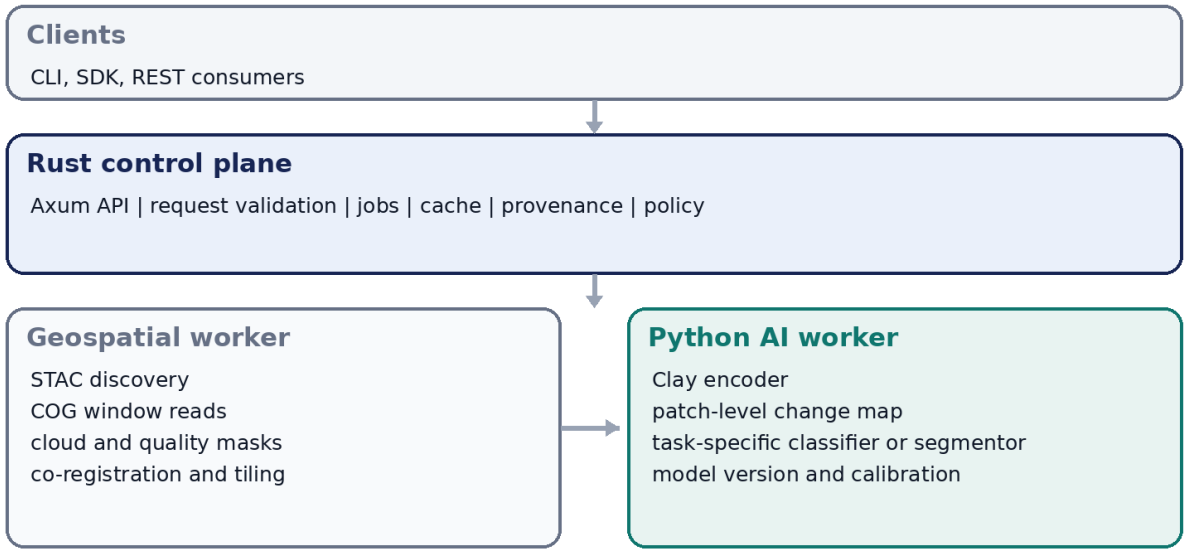
```
{
  "status": "review_recommended",
  "findings": [{
    "geometry": {"type": "Polygon", "coordinates": [...]},
    "change_score": 0.82,
    "suggested_class": "vegetation_loss",
    "class_probability": null,
    "changed_area_m2": 18420,
    "evidence": {"ndvi_delta": -0.31, "valid_pixel_ratio": 0.96}
  }],
  "observations": {
    "before_scene": "...", "after_scene": "...",
    "before_date": "2026-04-08", "after_date": "2026-06-10"
  },
  "model": {"encoder": "clay-v1.5", "change_method": "feature-diff-v0.1"},
  "artifacts": {"before": "...", "after": "...", "overlay": "..."}
}
```

Task registry instead of arbitrary questions

The public API should accept explicit, versioned tasks such as general_change, landcover, flood_extent, or vegetation_loss. A language layer may map user text into one of these tasks, but unsupported requests must be rejected rather than improvised.

6. Technical Architecture

Proposed MVP architecture



Docker Compose first. A native single-binary runtime is a later optimization, not an MVP promi

Why Rust remains valuable

Rust is appropriate for the control plane: a reliable API server, typed request contracts, concurrency, job orchestration, cache policy, provenance, and a future plugin interface. Rust does not need to own the first model runtime. Forcing PyTorch and the geospatial Python ecosystem into a native Rust runtime before product validation would add risk without proving user value.

Recommended MVP components

Component	Initial choice	Reason
API/control plane	Rust + Axum	Strong systems signal and reliable typed service
Raster processing	Python worker with Rasterio/GDAL or a proven service	Reduces geospatial implementation risk
AI inference	Python + PyTorch + Clay	Matches the supported model ecosystem
Catalog	STAC API adapters	Standardized scene discovery [3]
Raster access	COG range reads	Avoid full-tile downloads [7]
Metadata/cache	SQLite initially	Simple single-node deployment
Packaging	Docker Compose	Honest dependency boundary and reproducible startup
Artifacts	Local disk or S3-compatible store	Thumbnails, overlays, and exported rasters

Data handling

- Default input: Sentinel-2 Level-2A surface reflectance, avoiding an unnecessary atmospheric-correction subsystem. [6]

- Use date windows, not exact dates, because usable observations depend on revisit timing and local cloud conditions.
- Rank scenes using local valid-pixel quality over the area of interest, not only scene-level cloud percentage.
- Use scene-classification or quality bands to mask cloud, shadow, snow, and invalid pixels.
- Read COG windows and cache extracted analysis tiles rather than blindly caching whole satellite tiles.

7. Competitive Positioning

Oberon should not claim that open-source Earth observation APIs do not exist. STAC standardizes discovery, eoAPI provides reusable discovery and visualization infrastructure, openEO defines a processing API, and Raster Vision provides an ML framework. Clay itself provides the model and fine-tuning examples. [1][3][4][5]

The narrower differentiation

> *Oberon is the deployable inference and evidence layer between open imagery infrastructure and operational applications.*

Alternative	What it does well	Oberon differentiation
STAC	Catalog and search standard	Runs an opinionated analysis workflow after discovery
eoAPI / TiTiler	Search, visualization, raster access	Adds model inference, task registry, evidence, and evaluation
openEO	Interoperable EO processing graphs	Offers a focused self-hosted change-intelligence product
Raster Vision	Build and train geospatial ML pipelines	Provides a ready-to-query runtime and provenance contract
Clay	Foundation-model encoder and examples	Packages Clay into an operational API with data preparation and trust controls
Hosted geospatial AI	Convenient managed access	Local deployment, inspectable pipeline, and customer-controlled models

Defensibility, if the project earns it

- Task-quality moat: evaluated change models that work across regions and seasons.
- Workflow moat: reliable scene selection, co-registration, quality handling, and provenance.
- Feedback moat: customer-approved review labels improve specific models.
- Distribution moat: becoming the easiest open-source runtime for geospatial model deployment.
- Integration moat: adapters for STAC sources, model backbones, and downstream alert systems.

The implementation alone is not a durable moat. The durable value would come from reliability, evaluation, ecosystem adoption, and task-specific data.

8. MVP Scope and Evaluation

MVP definition

- One source: Sentinel-2 Level-2A.
- One analysis workflow: before/after general change triage.
- One model backbone: Clay v1.5.
- One deployment path: Docker Compose on a single machine.
- One response contract: findings, evidence, quality, artifacts, and provenance.
- One optional evaluated task head only if a suitable labeled dataset is selected.

Evaluation plan

Question	Metric	Why it matters
Did the system retrieve usable evidence?	valid-pixel ratio, date offset, scene-selection failure rate	Separates data failures from model failures
Does the change map rank real changes?	precision at K, recall at K, area overlap on labeled pairs	Tests triage usefulness
Can users verify results?	artifact completeness and reproducibility checks	Trust and debugging
Is latency operationally useful?	p50/p95 per area size and deployment profile	Capacity planning
Does AI beat a simple baseline?	comparison against spectral-only and pixel-difference baselines	Proves the model earns its complexity
Does it generalize?	geographic and temporal holdout tests	Avoids local overfitting

Release gate

> **Do not launch the "AI" claim based only on a visually impressive demo.**
> The first public release should publish a baseline comparison, known limitations, at least one geographically held-out evaluation, and examples where the system abstains or fails.

Success criteria for the first 10 users

- At least three users run the service without direct setup help.
- At least two users analyze more than one area or return for a second session.
- Users can explain when the output is useful and when they still need manual GIS work.
- At least one repeated task emerges strongly enough to justify a dedicated model head.
- The AI change method demonstrates measurable value over deterministic baselines.

9. Build Plan

Week	Deliverable	Decision reduced
0	Python vertical slice: STAC -> paired chips -> Clay features -> rough change map	Can the complete technical path work?
1	Scene selection and local quality scoring	Can Oberon reliably find usable evidence?
2	COG window reads, alignment, masks, and before/after artifacts	Can inputs be made analysis-ready?
3	Deterministic baselines: pixel, NDVI, NDWI, NBR deltas	What must AI beat?
4	Clay feature extraction and tiled feature-difference map	Does the foundation model add signal?
5	Rust Axum control plane and typed job contract	Can this become a reliable product API?
6	GeoJSON findings, provenance, model registry, and artifact store	Can results be verified and reproduced?
7	Evaluation dataset and geographic holdouts	Is the method useful beyond demos?
8	Docker Compose, CPU/GPU profiles, observability	Can others deploy and operate it?
9	CLI, SDK example, docs, and benchmark report	Can developers adopt it?
10	Design-partner launch and public technical write-up	Does anyone repeatedly need it?

Estimated effort

A credible part-time MVP is more likely to require roughly 120 to 180 focused hours than 74 hours. The uncertainty is concentrated in geospatial alignment, cloud handling, model evaluation, and deployment reproducibility. GPU spending can remain limited during prototyping if inference and experiments are batched, but responsive production inference should be treated as a separate deployment profile.

Build order principle

> *Prove the full data-to-model path in Python first, then build the Rust control plane around the proven workflow.*

The original orbital-pass CLI should be removed from the critical path. It teaches Rust, but it does not reduce risk for archival Sentinel analysis. Every early artifact should advance the final user workflow.

10. Deployment and Security Model

Deployment profiles

Profile	Runs where	Capability
Developer	Laptop or workstation	Small-area tests and batch inference
CPU self-hosted	General-purpose server	Slow batch analysis; no interactive-latency promise
GPU self-hosted	Single GPU host	Interactive or higher-throughput inference
Split deployment	Rust API on CPU + GPU worker	Scales inference independently
Disconnected package - later	Preloaded imagery, catalog, model, and signed artifacts	True offline operation after dedicated engineering

Terminology discipline

- Use local-first or self-hosted for the MVP.
- Use data-sovereign only when the deployment and data flow support that claim.
- Do not use air-gapped until models, imagery, catalogs, dependencies, updates, and verification can operate without external network access.
- Do not use zero data egress if the configured data source or telemetry sends information outside the environment.

Minimum production safeguards

- Signed container images and pinned model checksums.
- Software bill of materials and dependency scanning.
- Authentication, authorization, request limits, and audit logging for hosted deployments.
- Configurable telemetry disabled by default for self-hosted deployments.
- Model and dataset license records attached to every release.
- Resource limits and job isolation for untrusted requests.

11. Business Model and Validation

Open-source core

The core runtime can use Apache 2.0 if the goal is adoption and broad commercial use. Under that model, enterprise customers are not paying for permission to use the same core. They pay for convenience, support, operational guarantees, proprietary enterprise modules, custom models, or compliance work.

Offering	Potential value	Validation required
Open-source runtime	Adoption, credibility, integrations, contributors	Do developers deploy it and return?
Hosted API	No GPU or data-pipeline operations for the customer	Will users pay rather than self-host?
Enterprise support	SLAs, upgrades, troubleshooting, architecture guidance	Will organizations depend on it operationally?
Enterprise modules	SSO, audit, governance, model registry, offline packaging	Which requirements repeatedly block adoption?
Custom task models	Domain-specific training, evaluation, and deployment	Do customers own enough labels and budget?
Research or government grants	Fund a clearly matched technical objective	Must match an active solicitation and eligibility rules

Validation sequence

8. Technical proof: show AI improvement over simple baselines.
9. Problem proof: observe repeated manual review or monitoring pain.
10. Workflow proof: users integrate or repeatedly run the API.
11. Payment proof: charge for hosted convenience, support, or a task-specific capability.
12. Enterprise proof: only then invest in compliance and disconnected deployment.

What not to forecast yet

Do not tie GitHub stars directly to revenue, enterprise contracts, grant awards, or venture funding. Track evidence by stage: installations, repeat analyses, retained users, integrations, paid pilots, and operational dependency.

12. Risks and Decision Gates

Risk	Why it is real	Mitigation or gate
AI adds no value over indices	Simple spectral methods may solve the first task	Publish baseline comparisons; remove Clay if it does not win
Seasonal or atmospheric false changes	Different dates can look different without real land change	Season matching, masks, local quality metrics, abstention
No labeled data	Task-specific classification cannot be trusted	Start with change ranking; partner for a narrow labeled task
Rust slows delivery	Geospatial and ML tooling is Python-heavy	Rust control plane, Python workers, stable boundary
Self-host demand is weak	Many users prefer managed APIs	Offer hosted option only after usage evidence
Model output looks more certain than it is	Uncalibrated scores invite misuse	Explicit score semantics, calibration, evidence, review states
Scope expands into a platform too early	Multi-source protocol vision can consume years	No protocol work before repeat users and integrations
Public data resolution is insufficient	Sentinel-2 cannot resolve every object or event	State resolution limits and support commercial sources later

Decision gates

Gate 1 - after week 4
Does Clay-based change ranking outperform deterministic baselines on held-out examples? If not, keep Oberon as an imagery and measurement API or change the task.

Gate 2 - after week 8
Can an external user deploy the service and reproduce the benchmark? If not, fix packaging before adding features.

Gate 3 - after 10 users
Is there one repeated domain task worth a dedicated classifier or segmentor? If not, do not build a broad model catalog.

Gate 4 - before enterprise investment
Has a real buyer requested self-hosting, governance, or offline packaging and shown budget? If not, avoid speculative compliance work.

13. Expansion Path

Phase 2: evaluated task packs

Add versioned models for a small number of high-value tasks, each with its own dataset statement, metrics, thresholds, supported geographies, and known limitations. Candidate tasks include flood extent, vegetation loss, wildfire damage, and built-up expansion.

Phase 3: multi-source and customer models

Support Landsat, Sentinel-1 SAR, NAIP, and commercial providers through adapters. Add a model registry and bring-your-own-model contract. Multi-source support should follow demonstrated need, not precede it.

Phase 4: monitoring and alerting

Users register areas, schedules, and task thresholds. Oberon runs new-observation checks and sends evidence-backed alerts. This transforms a query API into an operational monitoring system and is likely more commercially valuable than one-off queries.

Phase 5: interoperable inference contract

If multiple operators and model providers adopt Oberon, formalize adapter and result schemas. A broader protocol is a possible outcome of adoption, not an MVP feature.

On-orbit inference

On-orbit or edge inference is a separate future market with distinct hardware, radiation, communications, mission, and certification constraints. The ground-based API may inform interface design, but it should not be used as evidence that an orbital product is near-term.

Long-term product vision

> *Become the open, inspectable runtime for turning Earth observation data into evaluated, evidence-backed machine findings wherever the data must be processed.*

14. Why This Project Is Worth Building

Career leverage

Even if the business does not materialize, Oberon can demonstrate systems thinking, geospatial data engineering, model integration, evaluation discipline, API design, open-source maintenance, and technical product judgment. That is a much stronger signal than merely wrapping an external AI API.

Learning leverage

- Rust for reliable services and concurrency.
- STAC, COG, coordinate reference systems, and raster processing.
- Foundation-model inference and feature extraction.
- Task-head training, evaluation, calibration, and abstention.
- Hybrid systems where deterministic methods verify learned models.
- Reproducible deployment and observability for compute-heavy workloads.

The disciplined thesis

> *Build the smallest system in which AI clearly reduces human review or produces a structured result that imagery alone cannot provide.*

Oberon should earn every expansion. First prove that the data pipeline works. Then prove that Clay improves a defined task. Then prove that users repeatedly need the workflow. Only after those gates should the project become a hosted service, enterprise product, protocol, or grant-funded company.

Recommended next action
Build a 2-3 day Python experiment that compares three change maps on the same before/after examples: raw pixel difference, spectral-index difference, and Clay feature difference. That experiment will reveal whether the proposed AI core deserves to exist.

15. Sources and Validation Notes

The brief intentionally separates supported technical facts from unvalidated product and business hypotheses. Sources below are primary project or standards documentation, accessed June 19, 2026.

[1] Clay Foundation Model - overview and downstream uses

[2] Clay v1.5 basic use - Sentinel-2 input and embedding example

[3] SpatioTemporal Asset Catalog specification

[4] eoAPI - reusable Earth observation API framework

[5] openEO - interoperable Earth observation processing API

[6] Copernicus Data Space - Sentinel-2 Level-2A documentation

[7] Cloud Optimized GeoTIFF - range-based partial reads

[8] Clay classification-head fine-tuning example

[9] Clay segmentation-head fine-tuning example

[10] Clay v1.5 model specification and known limitations

[11] Development Seed - Clay and stacchip data-pipeline context

Validation labels used in this brief

Label	Meaning
Supported	Primary documentation demonstrates technical availability or behavior.
Partially validated	Adjacent tools and evidence support the opportunity, but differentiation remains to be tested.
Unvalidated	Requires interviews, usage, pilots, or payment evidence.
Hypothesis	A strategic possibility, not a forecast or commitment.

This document supersedes Product Brief v2 for planning purposes. It removes unsupported certainty, narrows the MVP, corrects the role of Clay, and makes AI value measurable rather than assumed.